

Scrum Security Review Table

Scrum Ceremony/Artefact	Recommended Security Processes
1. Product Backlog	Identify security requirements as part of the user stories (Sharma & Bawa, 2022). - Initiate threat modelling to identify vulnerabilities early in the planning stage, such as SQL injection or insecure access controls (Stettina et al., 2023). - Technical Debt Management : Identify potential technical debt that may accumulate, such as temporary fixes for security issues (Aldaeej et al., 2023).
2. Sprint Planning	Integrate Security by Design principles into the sprint planning, ensuring that secure coding practices are followed (Sharma & Bawa, 2022). - Add security-specific tasks to the sprint backlog, such as implementing input validation or secure authentication mechanisms. - Plan for runtime patching if critical vulnerabilities are identified (Islam et al., 2023).
3. Daily Standup	Discuss any security incidents or newly discovered vulnerabilities so the team is aware of risks and remediation actions. - Check whether the previous day's security tests have passed, and whether immediate actions are required. - Implement Security Automation Tools to guarantee early detection of vulnerabilities.
4. Sprint Execution	Perform static analysis of the codebase (e.g., using SAST) to identify vulnerabilities such as insecure input handling (Stettina et al., 2023). - Apply secure coding practices such as parameterised queries and secure data encryption. - Plan for runtime patching : Identify whether there are patches

needed during the sprint for reported vulnerabilities (Islam et al., 2023).

- 5. Sprint Review** **Security review** of the delivered code and system functionalities to ensure compliance with predefined security requirements (Sharma & Bawa, 2022). - **Testing**: Conduct **dynamic analysis** (DAST) on staging environments to validate runtime security. - Discuss how **technical debt** has been addressed, such as patching previously identified vulnerabilities (Aldaej et al., 2023).
- 6. Sprint Retrospective** Perform a **post-mortem analysis** of any security incidents or missed vulnerabilities and their impact on the project. - Discuss which **security processes** worked well and which could be improved for the next sprint. - Plan for **Technical Debt**: Evaluate which technical debts were incurred and how they can be reduced in future sprints (Aldaej et al., 2023).

References

- Sharma, A. & Bawa, R.K. (2022) 'Identification and integration of security activities for secure agile development', *International Journal of Information Technology*, 14(1), pp. 1117-1130.
- Aldaej, A., Nguyen-Duc, A. & Gupta, V. (2023) 'A Lean Approach of Managing Technical Debt in Agile Software Projects – A Proposal and Empirical Evaluation', *Agile Processes in Software Engineering and Extreme Programming*, 475(1), pp. 67-76.
- Islam, C., Prokhorenko, V., & Babar, M.A. (2023) 'Runtime software patching: Taxonomy, survey and future directions', *Journal of Systems and Software*, 200(1), pp. 1-22.
- Stettina, C.J., Garbajosa, J. & Kruchten, P. (2023) 'Agile Processes in Software Engineering and Extreme Programming', 24th International Conference on Agile Software Development, 475(1), pp. 149-164.